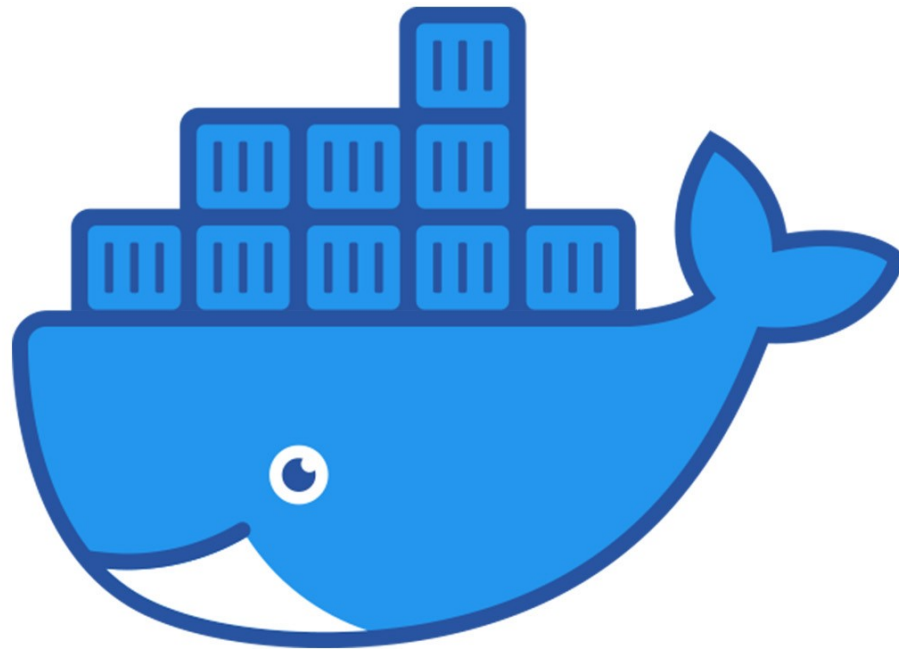


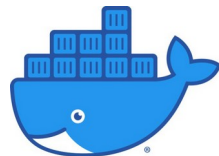


docker®

Prof. David Fernandes de Oliveira
Instituto de Computação
UFAM

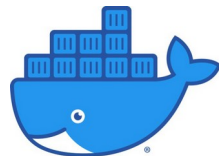
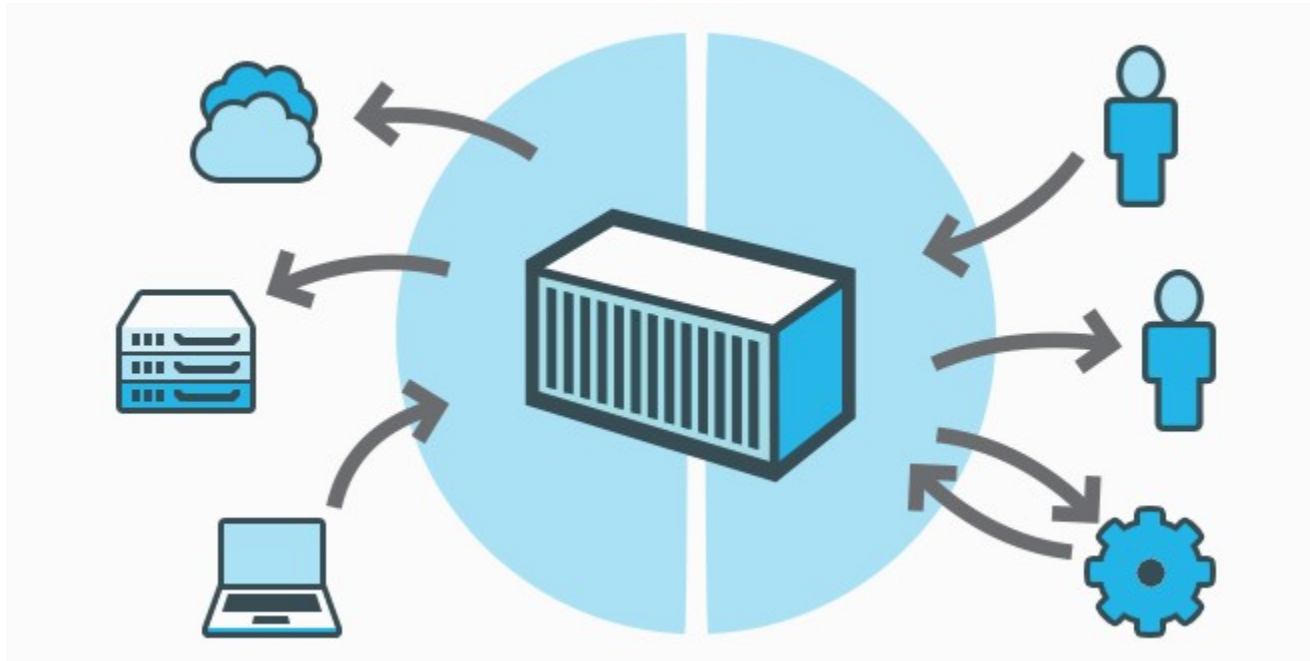
Pra que usar Docker?

- **O problema:** diferenças entre os ambientes de desenvolvimento, de testes e de produção
- O **Docker** permite empacotar um aplicativo com todas as suas dependências em **ambientes padronizados**
- Vantagens:
 - Ambientes semelhantes
 - Aplicação como pacote completo
 - Padronização e replicação



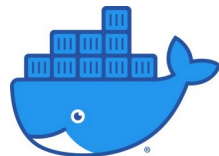
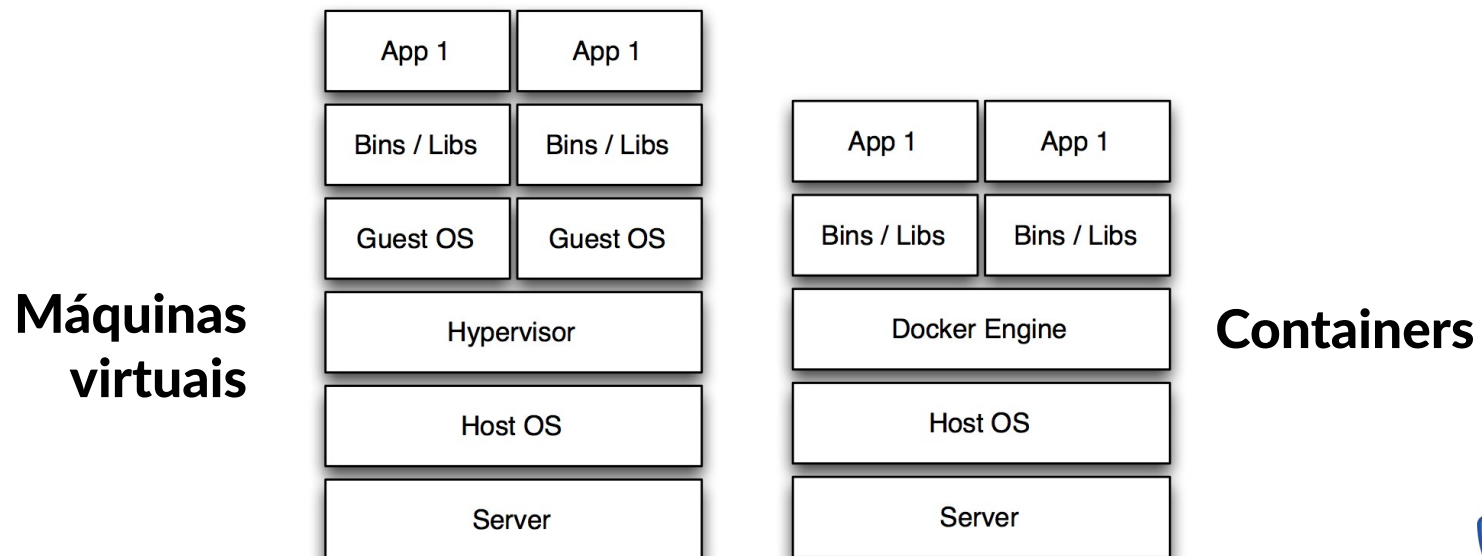
O que é o Docker?

- **Docker** é uma plataforma criada para facilitar o desenvolvimento, testes, implantação e execução de aplicações em **ambientes isolados e padronizados**



Containers × Máquinas Virtuais

- **Máquinas virtuais** virtualizam uma máquina completa
 - Cada VM possui seu próprio sistema operacional
 - Executadas sobre um **hypervisor**
- **Containers** isolam processos
 - Compartilham o kernel do sistema operacional host
 - Possuem seu próprio sistema de arquivos, processos e rede



Containers × Máquinas Virtuais

- **Máquinas virtuais** virtualizam uma máquina completa

Reproduzem um **sistema operacional** inteiro

possui seu próprio sistema operacional rodando sobre um **hypervisor**

Inicializam em milissegundos a poucos segundos

isolam processos

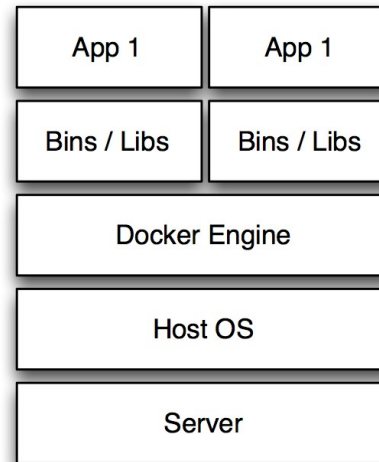
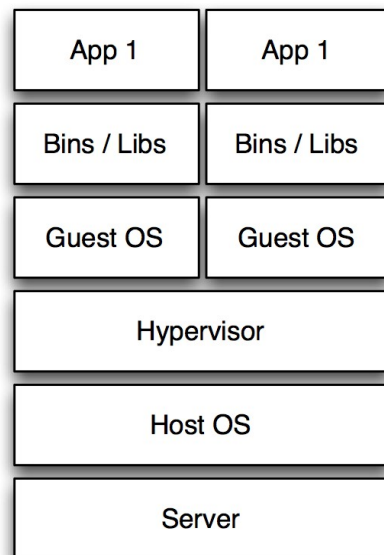
- Compartilham o kernel do sistema operacional do host

Necessitam de um **hypervisor**

possuem seu próprio sistema de arquivos, processo

Contêineres são apenas **processos isolados**, com overhead mínimo.

Máquinas virtuais
Mais pesadas e lentas para iniciar



Compartilha o kernel do sistema operacional com o host

Containers × Máquinas Virtuais

- **Máquinas virtuais** virtualizam uma máquina completa

Reproduzem um **sistema operacional** inteiro

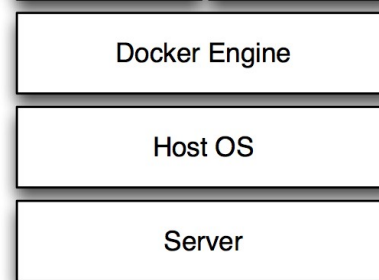
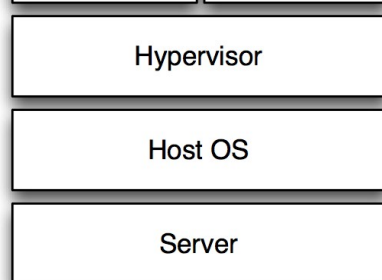
possui seu próprio sistema o das sobre um **hypervisor**

isolam processos

Inicializam em milissegundos a poucos segundos

```
david@coyote:~/dev/sn
Linux coyote 7.0.0-31-generic #31-Ubuntu SMP PREEMPT_DYNAMIC Sat Aug 1 04:26:38 UTC 2026 x86_64 GNU/Linux
david@coyote:~/dev/sn on main $ docker run nginx:1.31.6-alpine uname -a
Linux deb71ceebbfc 7.0.0-31-generic #31-Ubuntu SMP PREEMPT_DYNAMIC Sat Aug 1 04:26:38 UTC 2026 x86_64 Linux
david@coyote:~/dev/sn on main $
```

Máquinas
Mais pesadas e lentas para iniciar

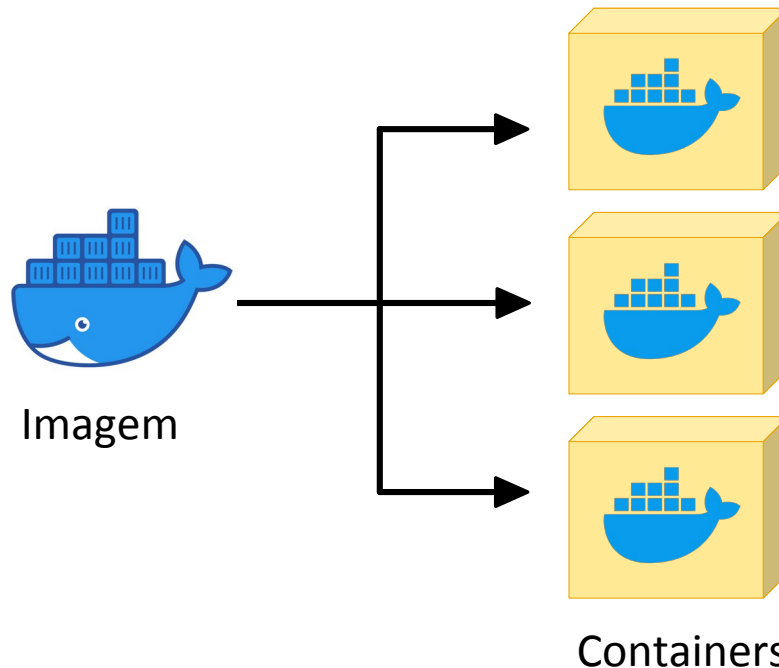


Compartilha o kernel do sistema operacional com o host

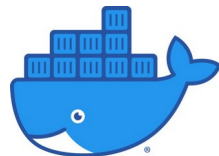
Imagem × Container

- Uma imagem Docker é um pacote imutável que contém o sistema de arquivos, a aplicação, bibliotecas e dependências necessárias para sua execução
- Um **container** é uma instância em execução de uma imagem

Uma **imagem** é um molde (template) para criação de containers. Contém os arquivos e dependências necessários para executar uma aplicação.



Um **container** é uma instância criada a partir de uma imagem, com seu próprio ambiente isolado de execução.



Docker Hub

The screenshot shows the Docker Hub search results for 'node.js'. The browser address bar shows 'hub.docker.com/search?q=node.js&badges=official'. The search results show two items: 'node' and 'mongo-express'. The 'node' item is a Docker Official Image, described as 'Node.js is a JavaScript-based platform for server-side and networking applications.' It has 2h, 1B+ downloads, and 10K+ stars. The 'mongo-express' item is also a Docker Official Image, described as 'DEPRECATED; Web-based MongoDB admin interface, written with Node.js and express'. It has 2y, 100M+ downloads, and 1.5K stars. On the left, there are filters for 'Products' (Images, Helm Charts, Sandbox Kits, AI Models, Compose) and 'Trusted content' (Docker Hardened Image, Docker Official Image, Verified Publisher). A blue box at the bottom right contains text explaining Docker Hub's function.

Explore Docker's Contain x +

hub.docker.com/search?q=node.js&badges=official

hub

node.js trusted-content:official CtrlK

Sign in Sign up

Filter by (1) [Clear All](#)

Products

- ☐ Images
- ☐ Helm Charts
- ☐ Sandbox Kits
- ☐ AI Models
- ☐ Compose

+ Show more

Trusted content

- ☐ Docker Hardened Image ⓘ
- ☒ Docker Official Image ⓘ
- ☐ Verified Publisher ⓘ

1 - 2 of 2 results for node.js.

Best match

DOCKEROFFICIAL IMAGE X

IMAGE + 1 MORE

node

Docker Official Image

Node.js is a JavaScript-based platform for server-side and networking applications.

2h 1B+ 10K+

IMAGE + 1 MORE

mongo-express

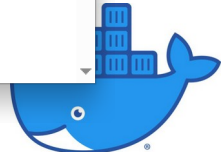
Docker Official Image

DEPRECATED; Web-based MongoDB admin interface, written with Node.js and express

2y 100M+ 1.5K

Give Feedback

Docker Hub é um repositório onde podemos baixar e publicar imagens Docker. Funciona como uma espécie de GitHub, só que focado em imagens Docker



Docker Hub

node - Official Image | Docker Hub

hub

node.js trusted-content:official CtrlK Sign in Sign up

Explore / Official Images / node

node Docker Official Image • 1B+ • 10K+

Node.js is a JavaScript-based platform for server-side and networking applications.

LANGUAGES & FRAMEWORKS

Overview Tags

Quick reference

- Maintained by:
[The Node.js Docker Team](#)
- Where to get help:
[the Docker Community Slack](#), [Server Fault](#), [Unix & Linux](#), or [Stack Overflow](#)

Supported tags and respective Dockerfile links

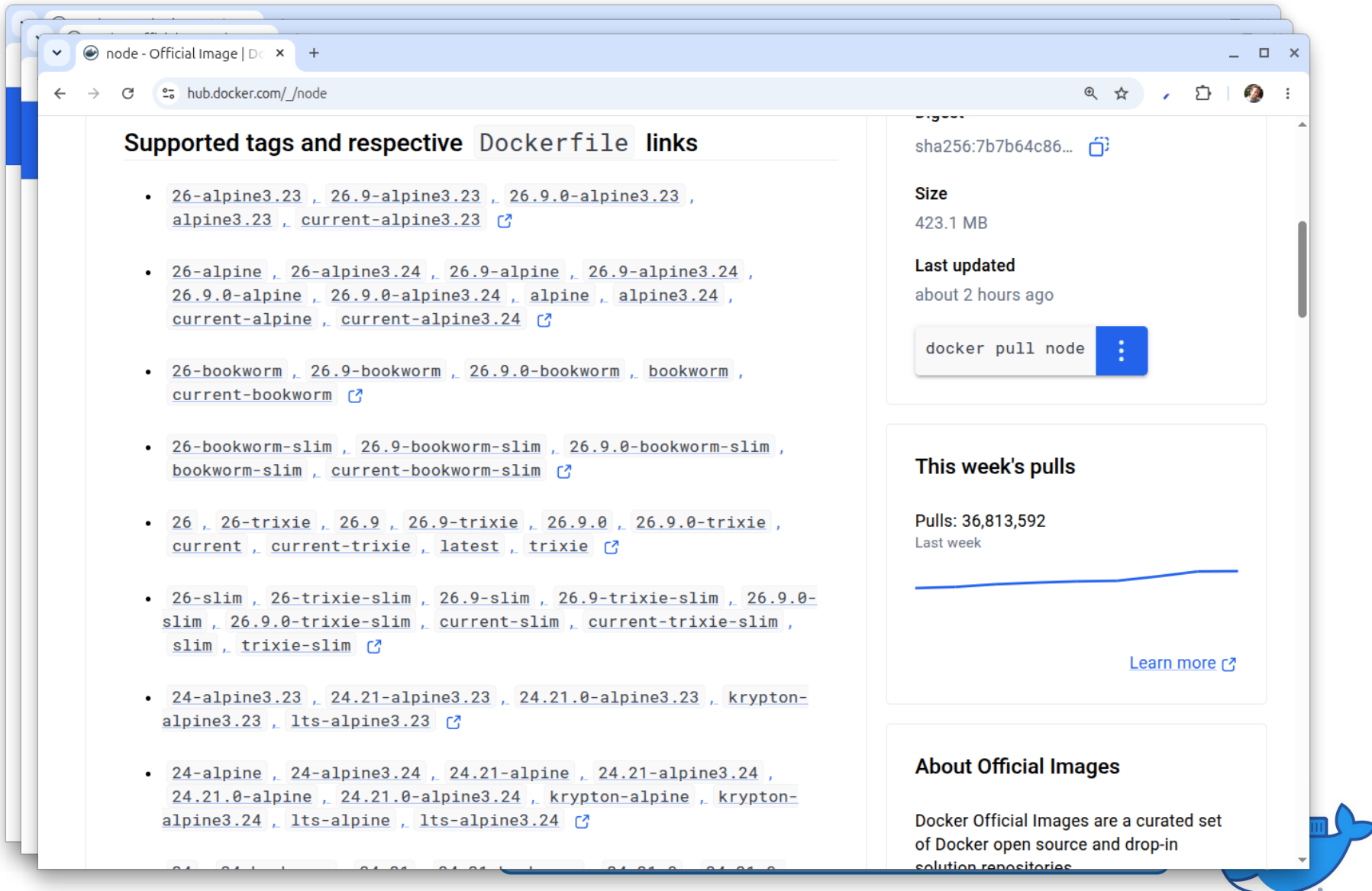
Tag summary

Recent tags
latest

Content type
Image

Digest
sha256:7b7b64c86...

Docker Hub



node - Official Image | Docker Hub

hub.docker.com/_/node

Supported tags and respective Dockerfile links

- [26-alpine3.23](#) , [26.9-alpine3.23](#) , [26.9.0-alpine3.23](#) , [alpine3.23](#) , [current-alpine3.23](#) [↗](#)
- [26-alpine](#) , [26-alpine3.24](#) , [26.9-alpine](#) , [26.9-alpine3.24](#) , [26.9.0-alpine](#) , [26.9.0-alpine3.24](#) , [alpine](#) , [alpine3.24](#) , [current-alpine](#) , [current-alpine3.24](#) [↗](#)
- [26-bookworm](#) , [26.9-bookworm](#) , [26.9.0-bookworm](#) , [bookworm](#) , [current-bookworm](#) [↗](#)
- [26-bookworm-slim](#) , [26.9-bookworm-slim](#) , [26.9.0-bookworm-slim](#) , [bookworm-slim](#) , [current-bookworm-slim](#) [↗](#)
- [26](#) , [26-trixie](#) , [26.9](#) , [26.9-trixie](#) , [26.9.0](#) , [26.9.0-trixie](#) , [current](#) , [current-trixie](#) , [latest](#) , [trixie](#) [↗](#)
- [26-slim](#) , [26-trixie-slim](#) , [26.9-slim](#) , [26.9-trixie-slim](#) , [26.9.0-slim](#) , [26.9.0-trixie-slim](#) , [current-slim](#) , [current-trixie-slim](#) , [slim](#) , [trixie-slim](#) [↗](#)
- [24-alpine3.23](#) , [24.21-alpine3.23](#) , [24.21.0-alpine3.23](#) , [krypton-alpine3.23](#) , [lts-alpine3.23](#) [↗](#)
- [24-alpine](#) , [24-alpine3.24](#) , [24.21-alpine](#) , [24.21-alpine3.24](#) , [24.21.0-alpine](#) , [24.21.0-alpine3.24](#) , [krypton-alpine](#) , [krypton-alpine3.24](#) , [lts-alpine](#) , [lts-alpine3.24](#) [↗](#)

sha256:7b7b64c86...

Size
423.1 MB

Last updated
about 2 hours ago

docker pull node [⋮](#)

This week's pulls

Pulls: 36,813,592
Last week



[Learn more](#) [↗](#)

About Official Images

Docker Official Images are a curated set of Docker open source and drop-in solution repositories.

Comando Docker Pull

- O comando **docker pull** pode ser usado para baixar uma imagem do docker hub

```
david@coyote:~/dev/sn  
david ~/dev/sn on  main $ docker pull nginx:1.31.6-alpine  
1.31.6-alpine: Pulling from library/nginx  
e2de96513ba9: Pull complete  
fb08ee61b60d: Pull complete  
f408df9a891a: Pull complete  
873bdb7a78a1: Pull complete  
90de1d15efe2: Pull complete  
cb2f2cc9a341: Pull complete  
ce3e77cbba4d: Pull complete  
c0ac6b3b9c77: Pull complete  
9d8ea080e87f: Download complete  
ec6129a9436d: Download complete  
Digest: sha256:adad2ae9204d0fd7a34f40299bc838c3782be1293b10005eac4315ff5a1abf4e  
Status: Downloaded newer image for nginx:1.31.6-alpine  
docker.io/library/nginx:1.31.6-alpine  
david ~/dev/sn on  main $
```

Nginx é um servidor web leve e de alto desempenho que também atua como proxy reverso e balanceador de carga

NGINX

took 14s



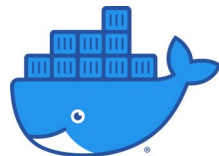
Tags × Digest

- Uma **tag** é um nome conveniente associado a uma imagem

```
$ docker pull nginx:1.31.6-alpine
```

- Mas tags são mutáveis e o mantenedor pode publicar novamente a mesma tag apontando para outro conteúdo
- O **digest** identifica exatamente o conteúdo da imagem:

```
$ docker pull nginx:sha256:adad2ae9204d0fd7a34f...
```



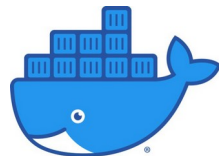
Comando Docker Images

- O comando **docker images** é usado para listar as imagens disponíveis no computador local

```
david@coyote:~/dev/sn  
david ~/dev/sn on main $ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	In Use
hello-world:latest	c3cbe1cc1aa5	25.9kB	9.49kB	U
minio/minio:latest	14cea493d9a3	241MB	62.2MB	
mysql:latest	66aec17cd21a	1.3GB	290MB	U
nginx:1.31.6-alpine	adad2ae9204d	94.4MB	27.2MB	
valkey/valkey:8.1.6	1f84517eca8e	163MB	43MB	

```
david ~/dev/sn on main $
```



Comando Docker Images

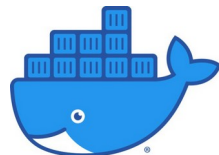
- O comando **docker images** é usado para listar as imagens disponíveis no computador local

```
david@coyote:~/dev/sn
~/dev/sn
david ~/dev/sn on main $ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	U In Use
nginx:1.31.6-alpine	3-b1c1c1c5	25.8MB	9.48MB	
nginx:1.31.6-alpine	4442dc9204d	94.4MB	27.2MB	
valkey/valkey:8.1.6	1f84517eca8e	163MB	43MB	

Para remover imagens, pode-se usar o comando **docker image rm <imagem>**. Ex: **docker image rm nginx:1.31.6-alpine**

```
david ~/dev/sn on main $
```

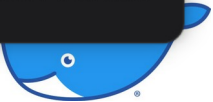


Comando Docker Run

- O comando **docker run** é usado para criar e inicializar um container a partir de uma imagem

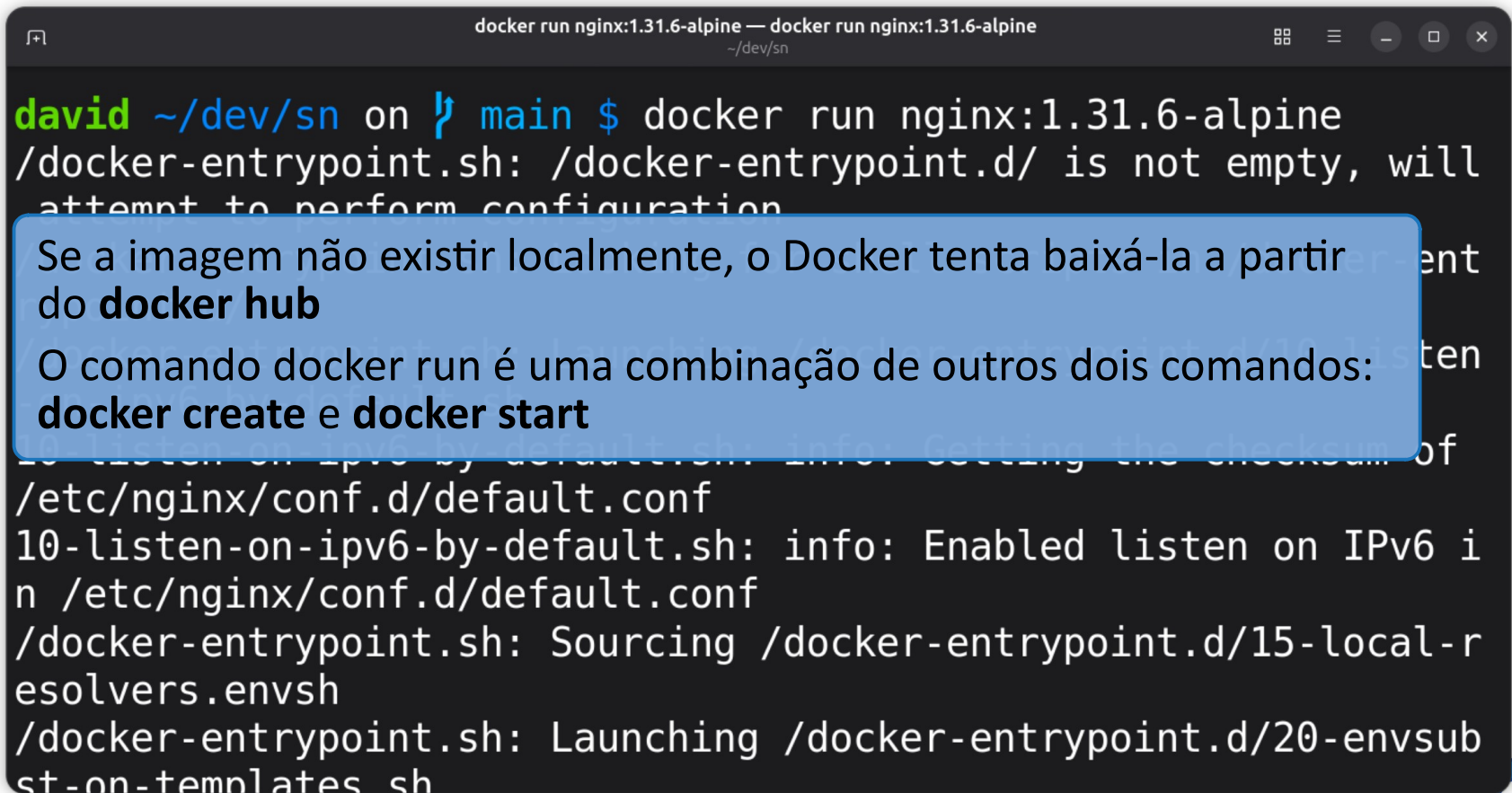
```
docker run nginx:1.31.6-alpine — docker run nginx:1.31.6-alpine
~/dev/sn

david ~/dev/sn on  main $ docker run nginx:1.31.6-alpine
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will
attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-ent
rypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen
-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of
/etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 i
n /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-r
esolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsub
st-on-templates.sh
```



Comando Docker Run

- O comando **docker run** é usado para criar e inicializar um container a partir de uma imagem



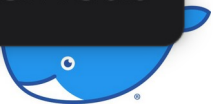
```
docker run nginx:1.31.6-alpine — docker run nginx:1.31.6-alpine
~/dev/sn

david ~/dev/sn on  main $ docker run nginx:1.31.6-alpine
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will
attempt to perform configuration

Se a imagem não existir localmente, o Docker tenta baixá-la a partir
do docker hub

O comando docker run é uma combinação de outros dois comandos:
docker create e docker start

/docker-entrypoint.sh: info: Getting the checksum of
/etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 i
n /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-r
esolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsub
st-on-templates.sh
```



Comando Docker Run

- Por padrão, o comando **docker run** bloqueia o uso do terminal
- Podemos usar **docker run** com a opção **-d, detach**, para que o terminal seja liberado após a execução do comando

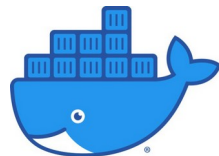
```
david@coyote:~/dev/sn
david ~/dev/sn on  main $ docker run -d nginx:1.31.6-alpine
35e21c1825cbacdd3a28287c6fc69fad2562c03d69bc03f1274ccf5562903367
david ~/dev/sn on  main $ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED
STATUS        PORTS               NAMES
35e21c1825cb   nginx:1.31.6-alpine  "/docker-entrypoint..." 4 seconds ago
Up 3 seconds    80/tcp             gifted_kare
723796425cef   mysql:latest         "docker-entrypoint.s..." 4 weeks ago
Up 3 days      33060/tcp, 0.0.0.0:3316->3306/tcp, [::]:3316->3306/tcp    smartnotes-db
david ~/dev/sn on  main $ ;5~
```



Comando Docker Run

- Um novo container é criado cada vez que executamos o comando **docker run [imagem]:[tag]**
- Desta forma, os comandos abaixo criam dois containers diferentes, ambos baseados na mesma imagem

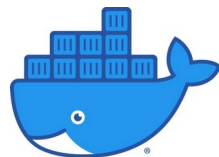
```
david@coyote:~/dev/sn
david ~/dev/sn on  main $ docker run -d nginx:1.31.6-alpine
052941520f9a76523b65c18c08212fef36d5b7bc21f0cbb314817d5180722c8b
david ~/dev/sn on  main $ docker run -d nginx:1.31.6-alpine
96afdf0cfe9cf39d35f143e937a5ae6b7767e2c6fe49b0b4711c9a8ee47404a7
david ~/dev/sn on  main $
```



Processo principal do container

- O processo principal de um container é o processo que o Docker inicia como PID 1 dentro do container e cujo ciclo de vida determina o ciclo de vida do próprio container
- Apesar de cada imagem possuir um comando principal padrão para inicialização do container, podemos mudar esse comando usando o **docker run**

```
david@coyote:~/dev/sn
david ~/dev/sn on ↳ main $ docker run nginx:1.31.6-alpine echo "Olá"
Olá
david ~/dev/sn on ↳ main $
```

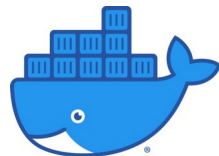


Terminais interativos

- Usamos o comando **docker run** com a opção **-it** para que o docker disponibilize um terminal interativo

```
docker run -it node:26.9.0-alpine3.23 — docker run -it node:26.9.0-alpine3.23
~/dev/sn
david ~/dev/sn on 1 main $ docker run -it node:26.9.0-alpine3.23
Welcome to Node.js v26.9.0.
Type ".help" for more information.
>
```

- Caso o comando **docker run** seja usado e a imagem não esteja disponível, a imagem será baixada automaticamente



Comandos Básicos

- O comando **docker ps** pode ser usado para visualizar os containers que estão sendo executados no momento

```
david ~/dev/sn on  main $ docker ps
CONTAINER ID   IMAGE                                COMMAND                                CREATED
35e21c1825cb   nginx:1.31.6-alpine                "/docker-entrypoint..."            3 hours
723796425cef   mysql:latest                        "docker-entrypoint.s..."           4 weeks
david ~/dev/sn on  main $
```

- Podemos parar a execução de um container através do comando **docker stop <container id>**

```
david@coyote:~/dev/sn
david ~/dev/sn on  main $ docker stop 35e21c1825cb
35e21c1825cb
david ~/dev/sn on  main $
```

Comandos Básicos

- Podemos usar **docker ps -a** para ver os containers parados

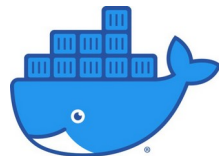
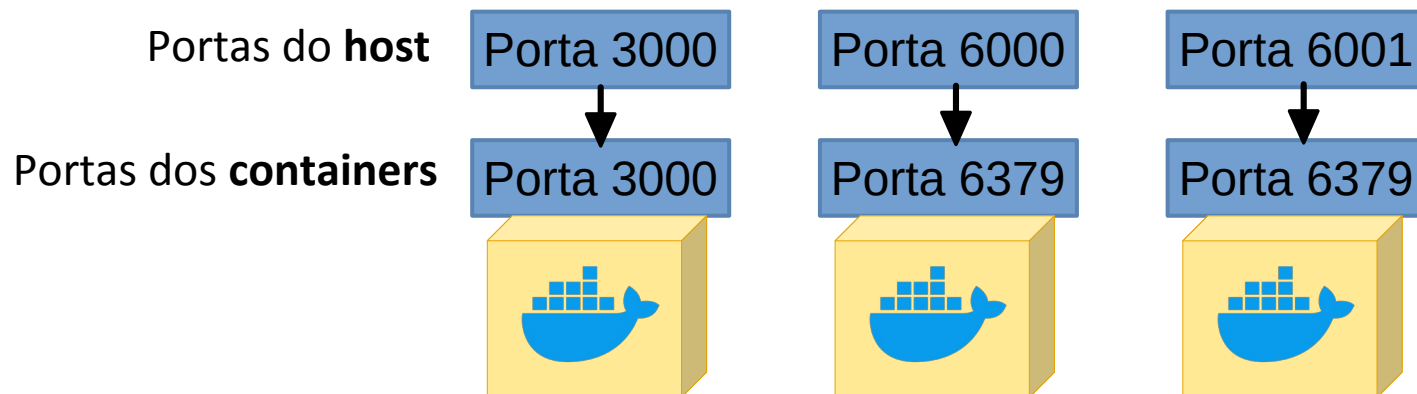
```
david ~/dev/sn on 3 main $ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                                CREATED
4a0f0dcedd8d   node:26.9.0-alpine3.23             "docker-entrypoint.s..."           21 minutes
35e21c1825cb   nginx:1.31.6-alpine                "/docker-entrypoint..."           3 hours
723796425cef   mysql:latest                        "docker-entrypoint.s..."           4 weeks
a00cf14b6421   hello-world                          "/hello"                             2 months
david ~/dev/sn on 3 main $
```

- Para reiniciar um container, usamos **docker start <container>**

```
david ~/dev/sn on 3 main $ docker start 35e21c1825cb
35e21c1825cb
david ~/dev/sn on 3 main $
```

Mapeamento de Portas

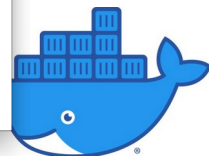
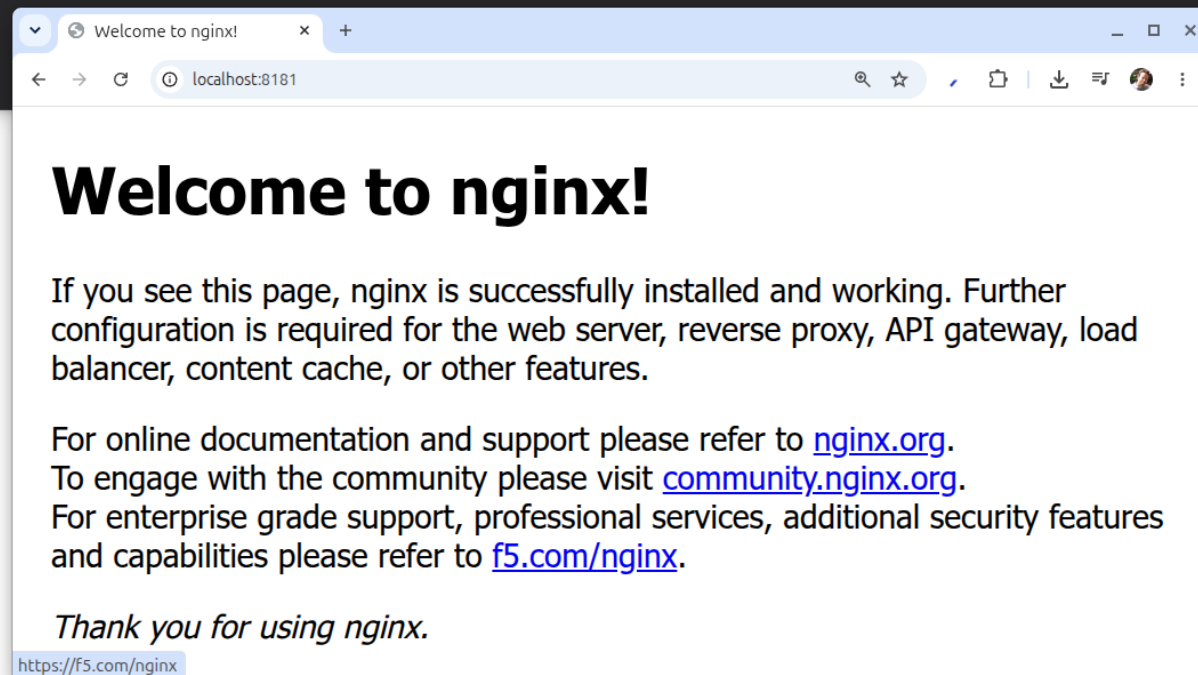
- Um servidor (host) pode rodar múltiplos containers em um dado momento, mesmo containers de uma mesma imagem
- Dependendo do serviço oferecido pelo container, ele precisa ser acessado através de uma porta
- Para acessar o container através de uma porta, é preciso que essa porta seja mapeada (bind) a uma dada porta do host



Mapeamento de Portas

- Para mapear a porta do container a uma porta do host, usamos o comando **docker run** com a opção **-p**

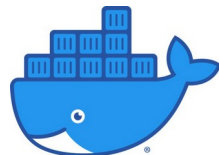
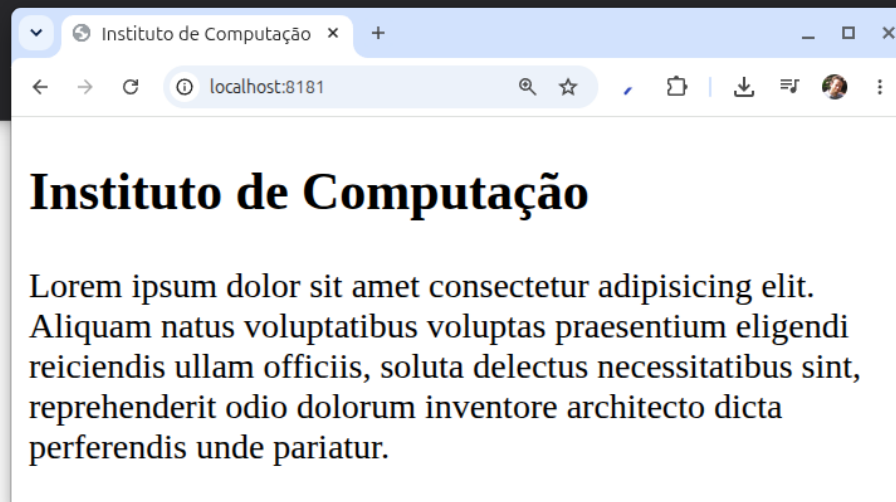
```
david@coyote:~/dev/sn  
david ~/dev/sn on  main $ docker run -d -p8181:80 nginx:1.31.6-alpine  
f5acd2b5acbbf545e509a9c888291b31f0738745248ff82c2b2653bca4fae3ee  
david ~/dev/sn on  main $
```



Mapeando volumes

- Usando **docker run**, também é possível montar diretórios de sua máquina local dentro dos containers através da opção **-v**
- O Nginx serve conteúdo por padrão em **/usr/share/nginx/html**, então basta montar sua pasta local nessa rota usando **-v**

```
david@coyote:~/dev/html
david ~/dev/html $ docker run -d -p8181:80 -v$(pwd)/site:/usr/share/nginx/html:ro nginx:1.31.6-alpine
5af3e9fb8670a9bd54bd5bb34201d10a5eb9aa8cd904228860bc1e038a73cb45
david ~/dev/html $
```



Verificando os logs

- O comando **docker logs** retorna todos os dados enviados para STDOUT e STDERR do container

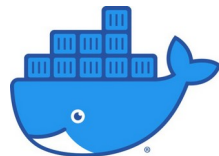
```
david@coyote:~/dev/html
david ~/dev/html $ docker logs -n3 5af3e9fb
172.17.0.1 - - [20/Sep/2026:14:15:38 +0000] "GET / HTTP/1.1"
 200 569 "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/53
7.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36" "-"
172.17.0.1 - - [20/Sep/2026:14:15:38 +0000] "GET /favicon.ic
o HTTP/1.1" 404 555 "http://localhost:8181/" "Mozilla/5.0 (X
11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chr
ome/153.0.0.0 Safari/537.36" "-"
2026/09/20 14:15:38 [error] 30#30: *1 open() "/usr/share/ngi
nx/html/favicon.ico" failed (2: No such file or directory),
client: 172.17.0.1, server: localhost, request: "GET /favico
n.ico HTTP/1.1", host: "localhost:8181", referer: "http://l
ocalhost:8181/"
david ~/dev/html $
```



Executando comandos

- Outra ferramenta útil para debug dos containers é o **docker exec**, que permite acessar os containers através de um shell

```
docker exec -it 30b5c24666ea /bin/sh
david@coyote ~/dev/pw $ docker exec -it 30b5c24666ea /bin/sh
/ # ls
bin                home               proc               sys
dev                lib                root               tmp
docker-entrypoint.d media              run                usr
docker-entrypoint.sh mnt               /sbin               var
etc                opt                srv
```



Exercício

- Crie uma pasta chamada site e, dentro dela, um arquivo index.html contendo um código HTML simples
- Em seguida, crie um container que satisfaça os seguintes requisitos:
 - utilize a imagem nginx:alpine
 - tenha o nome site
 - execute em segundo plano
 - publique a porta 8080 do host para a porta 80 do container
 - monte a pasta local site/ em /usr/share/nginx/html
 - monte o diretório como somente leitura
- Ao final, acesse <http://localhost:8000> e verifique se a página criada é exibida



Exercício

- Com o container em execução, utilize:

```
$ docker ps
```

- Identifique o **nome do container**, a **imagem utilizada**, o **estado do container** e o **mapeamento de portas**
- Depois, utilize o comando abaixo e localize as informações referentes ao **volume montado**:

```
$ docker inspect site
```

- Em seguida, entre no container:

```
$ docker exec -it site sh
```

- E veja o conteúdo do arquivo
`/usr/share/nginx/html/index.html`

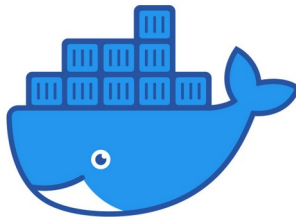
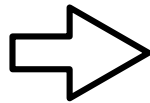


O Dockerfile

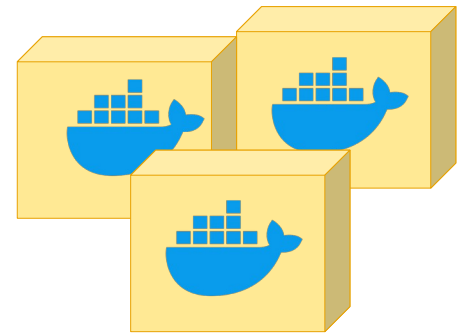
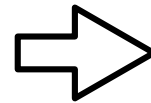
- O **Dockerfile** é um arquivo onde podemos definir diretrizes para criarmos nossas próprias imagens Docker
 - Essas diretrizes podem conter, por exemplo, instalação de pacotes, criação de diretórios, definição de variáveis de ambiente, etc



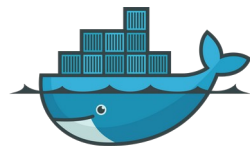
Dockerfile



Imagem



Containers



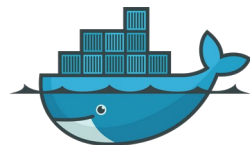
O Dockerfile

- Podemos exemplificar o uso do Dockerfile através de uma app express/typescript, tal como a mostrada abaixo

```
import express from "express"
const app = express();
app.get("/", (req, res) => {
  res.send("Hello World!");
});
app.listen(3000);
```

```
~/d/docker
david@coyote ~/dev/docker $ tree
.
├── package.json
└── src
    └── index.js

1 directory, 2 files
david@coyote ~/dev/docker $
```



Um possível Dockerfile para a App

```
FROM node:22-alpine
WORKDIR /app

# Instalação das dependências
COPY package*.json ./
RUN npm install

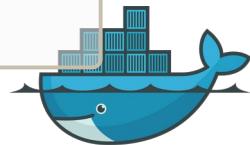
# Copia todo o código da aplicação
COPY . .

# Compila e move o código compilado para ./dist
RUN npm run build

# Informa que o ambiente é de produção
ENV NODE_ENV=production

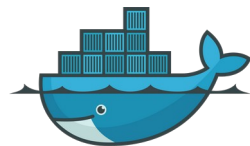
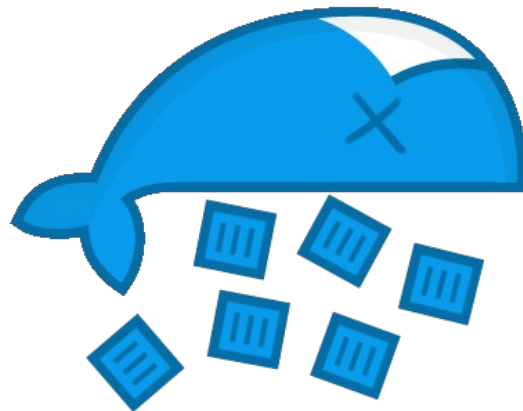
# Expõe a porta
EXPOSE 3000

# Roda a aplicação
CMD ["npm", "run", "start:prod"]
```



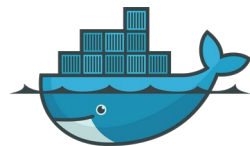
O problema do Dockerfile anterior

- No entanto, o Dockerfile anterior é muito ruim. Por quê?
 - Leva **devDependencies** e ferramentas de build para produção
 - Não usa **npm ci** nem trava versões com o lockfile
 - Pode vaziar **node_modules** e **.env** sem **.dockerignore**
 - Imagem maior, boot mais lento e deploys mais pesados
 - Serve apenas para **produção**, não para **desenvolvimento** (sem nodemon)



Multi-stage building

- Vários **estágios** (FROM ... AS nome)
 - Você pode criar quantos estágios quiser dentro de um único Dockerfile. Ex: build, test, production, development
- Cada estágio contém apenas o necessário
 - **build**: transpiler, bundler
 - **production**: só artefatos finais + dependências de runtime
 - **development**: com nodemon, testes e devDeps.
- Dá para mover só os artefatos necessários de um estágio para outro (COPY --from=build ...)
- Resultado: imagens menores e seguras, deploys mais rápidos



Multi-stage building

Dockerfile

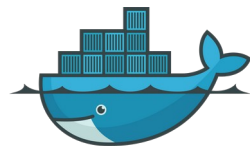
```
FROM node:22-alpine3.22 AS build
WORKDIR /app
COPY package*.json .
RUN npm ci
COPY . .
RUN npm run build

FROM node:22-alpine3.22 AS production
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json .
RUN npm ci --omit=dev
COPY --from=build /app/dist ./dist
EXPOSE 3000
CMD ["npm", "run", "start:prod"]

FROM node:22-alpine3.22 AS development
WORKDIR /app
ENV NODE_ENV=development
COPY package*.json .
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

.dockerignore

```
node_modules
.git
.env
coverage
dist
Dockerfile
docker-compose.yml
```



Dockerfile

```
FROM node:22-alpine3.22 AS build
```

```
WORKDIR /app
```

```
ENV NODE_ENV=development
```

```
COPY package*.json .
```

```
RUN npm install
```

```
COPY . .
```

```
EXPOSE 3000
```

```
CMD ["npm", "start"]
```

.dockerignore

```
node_modules
```

```
.env docker:default
```

```
coverage 0.0s
```

```
0.0s
```

```
0.3s
```

```
0.0s
```

```
0.0s
```

```
0.0s
```

```
0.0s
```

```
0.0s
```

```
2.4s
```

```
0.0s
```

```
0.3s
```

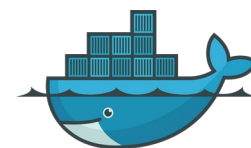
```
0.3s
```

```
0.0s
```

```
0.0s
```

```
took 3s
```

Após a criação do **Dockerfile**, usamos o comando `docker build` para construir a imagem no stage desejado.



Dockerfile

.dockerignore

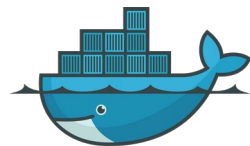
```
david@coyote ~/dev/pw/projeto $ docker build --target development -t minha-app:dev .
[+] Building 3.0s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 499B
=> [internal] load metadata for docker.io/library/docker:default
=> [internal] load dockerignore
```

Após a criação do

```
david@coyote ~ $ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
minha-app	dev	573982fbf047	5 minutes ago	212MB
<none>	<none>	836ccbed43cb	8 minutes ago	164MB
<none>	<none>	fc1713da25ff	4 hours ago	164MB
<none>	<none>	07fb53458b7d	4 hours ago	164MB
<none>	<none>	a95726cff3f2	5 hours ago	212MB
<none>	<none>	da006a279db8	19 hours ago	212MB
node	24.6.0-alpine3.22	b15a96093dc9	4 days ago	168MB
nginx	1.29-alpine	4a86014ec699	5 days ago	52.5MB
sandbox	latest	b2c871ff5ac1	11 days ago	3.57GB
redis	latest	2f21f6e6fe03	2 weeks ago	157MB
ubuntu	22.04	1d3ca894b30c	5 weeks ago	77.9MB
phpmyadmin/phpmyadmin	latest	0276a66ce322	6 months ago	571MB
hello-world	latest	74cc54e27dc4	6 months ago	10.1kB
mysql	8.4.4	4a8a163431d3	6 months ago	769MB
redis/redis-stack-server	7.2.0-v9	bc3453e3b2f5	17 months ago	467MB

```
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```



Dockerfile

.dockerignore

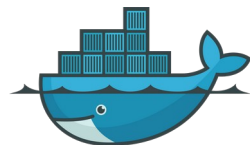
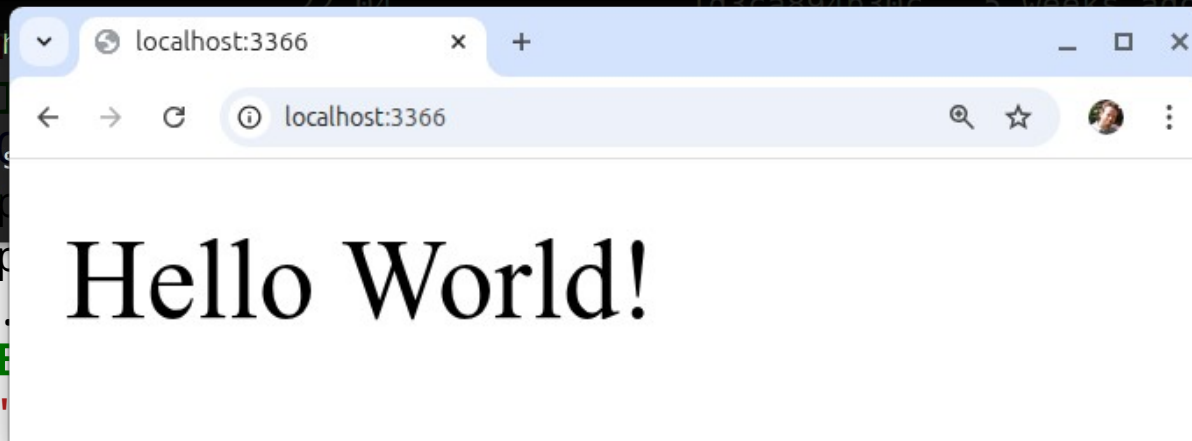
```
FROM node:22-alpine3.22 AS build
WORKDIR /app
COPY package.json ./
RUN npm install
COPY . .
EXPOSE 3333
CMD ["npm", "start"]
```

```
docker run -v $(pwd):/app -v app-node_modules:/app/node_modules -p3366:3333
```

```
david@coyote ~/dev/pw/projeto $ docker run -v $(pwd):/app -v app-
node_modules:/app/node_modules -p3366:3333 minha-app:dev
```

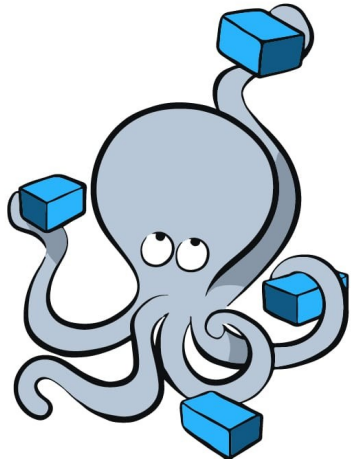
```
> projeto@1.0.0 start
> nodemon --watch src --exec tsx src/index.ts
```

```
[nodemon] 3.1.10
[nodemon] to restart at any time, enter rs
[nodemon] watching path(s): src/**/*.js
[nodemon] watching extensions: ts,json
[nodemon] starting `tsx src/index.ts`
Server running on port 3333
```

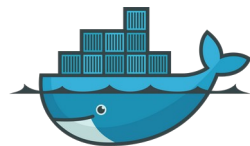


Docker Compose

- **O Docker Compose** é uma ferramenta utilizada para containerizar aplicações que requerem mais de um container
 - Ex: Apps com backend, frontend e um banco de dados
- Através do Docker Compose, podemos criar um arquivo YAML para definir e configurar cada serviço usado na aplicação
- Uma vez criado o arquivo YAML, podemos iniciar e parar todos os serviços da aplicação com um único comando

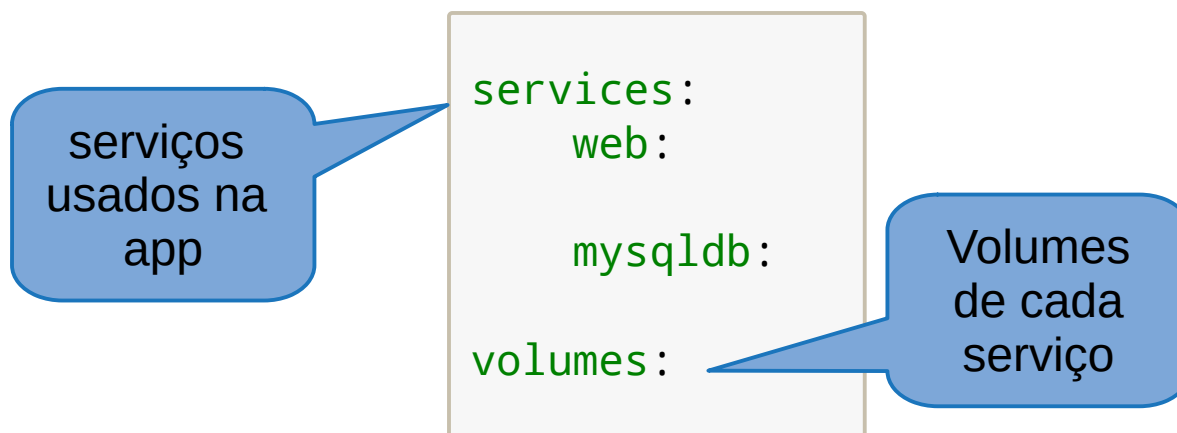


docker
Compose



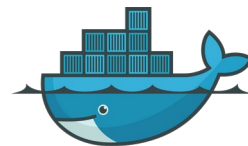
Docker Compose

- O primeiro passo para usar o **compose** é criar um arquivo **docker-compose.yml** na pasta raiz da aplicação



Volumes

- Os volumes provêem uma forma de **persistir os dados** usados/criados pelos serviços de uma aplicação
- Através dos volumes é possível mapear um diretório interno do container a um diretório da máquina host
 - Isso permite salvar os dados e as alterações feitas durante a execução do contêiner
- Por exemplo, sem o uso de volumes, todos os registros que o aplicativo armazena no banco de dados serão perdidos quando o contêiner parar ou reiniciar
 - Para evitar isso, temos que definir os volumes na configuração do serviço de banco de dados

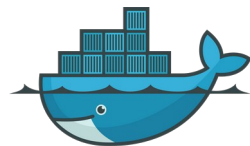


Volumes

- O **Docker Compose** oferece várias maneiras de definir volumes

```
volumes:  
  # Deixa o Docker criar um volume no sistema local  
  - /var/lib/mysql  
  
  # Mapena o volume do host no container  
  - ./opt/data:/var/lib/mysql  
  
  # Volume com nomes  
  - datavolume:/var/lib/mysql
```

- Embora os volumes sejam definidos em cada serviço, temos que listar os volumes na seção de volumes de nível superior



Arquivo de Ambiente

- Antes de criar o arquivo YAML, podemos criar um arquivo .env com as variáveis de ambiente usadas no código do aplicativo

```
// .env
```

```
MYSQL_ROOT_PASSWORD=root
```

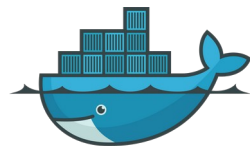
```
MYSQL_DATABASE=store
```

```
MYSQL_LOCAL_PORT=3306
```

```
MYSQL_DOCKER_PORT=3306
```

```
NODE_LOCAL_PORT=3001
```

```
NODE_DOCKER_PORT=3000
```



Configurando o serviço Web

- Agora, é hora de configurar o primeiro dos dois serviços em nosso aplicativo, o serviço da web

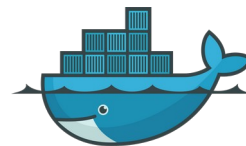
```
web:
  build:
    context: .
    env_file: ../.env
    command: npm start
    volumes:
      - ../app/
      - /app/node_modules
  ports:
    - $NODE_LOCAL_PORT:$NODE_DOCKER_PORT
  depends_on:
    - mysqlldb
  environment:
    MYSQL_HOST: mysqlldb
```

Informa o path do Dockerfile

Informa que serviços a App depende

Informa o path do .env

Retorna o IP do servidor MySQL

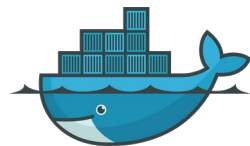


Configurando o serviço do Banco

- As opções de serviço de banco de dados seguem um padrão semelhante ao serviço da web

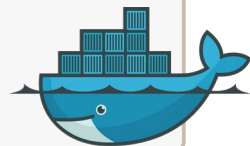
Define a
imagem
usada

```
mysqlldb:  
  image: mysql  
  env_file: ../.env  
  environment:  
    MYSQL_ROOT_PASSWORD: $MYSQL_ROOT_PASSWORD  
    MYSQL_DATABASE: $MYSQL_DATABASE  
    MYSQL_ROOT_HOST: "%"  
  ports:  
    - $MYSQL_LOCAL_PORT:$MYSQL_DOCKER_PORT  
  volumes:  
    - mysql:/var/lib/mysql  
    - mysql_config:/etc/mysql
```



Ao final, este é o
arquivo **docker-
compose.yml**
final que criamos
para nosso
aplicativo.

```
version: '3.8'
services:
  web:
    build:
      context: .
    env_file: ./env
    command: npm start
    volumes:
      - ./usr/app
    ports:
      - $NODE_LOCAL_PORT:$NODE_DOCKER_PORT
    depends_on:
      - mysql
    environment:
      MYSQL_HOST: mysql
  mysql:
    image: mysql
    env_file: ./env
    environment:
      MYSQL_ROOT_PASSWORD: $MYSQL_ROOT_PASSWORD
      MYSQL_DATABASE: $MYSQL_DATABASE
      MYSQL_ROOT_HOST: "%"
    ports:
      - $MYSQL_LOCAL_PORT:$MYSQL_DOCKER_PORT
    volumes:
      - mysql:/var/lib/mysql
      - mysql_config:/etc/mysql
volumes:
  mysql:
  mysql_config:
```



Subindo os serviços

- Uma vez que o docker-compose.yml esteja pronto, podemos iniciar os serviços através do comando docker-compose

```
$ docker compose up
```

- Para pararmos os serviços da aplicação, usamos o docker-compose com a opção down

```
$ docker compose down
```